

Guía de Ejercicios de Programación Orientada al Objeto para Técnicas Computacionales para Radio Interferometría

Metodología sugerida

Cada bloque introduce el concepto con un mini-ejemplo del dominio de la radio interferometría. Sigue los pasos para reproducir el ejemplo, resuelve los ejercicios y explica tus decisiones en una bitácora técnica.

1. Creación y destrucción de objetos

Contexto: administrar un interferómetro que crea y destruye antenas bajo demanda sin replicar estructuras de datos innecesarias.

1. Define **Interferometro** como fachada sobre una lista interna de antenas. Al instanciarlo, recibe una secuencia de diccionarios con metadatos y crea objetos **Antena** (con `__init__` que toma `id`, `longitud`, `latitud`) sólo dentro del interferómetro. Registra en consola cada reserva de calibración para confirmar que los objetos viven dentro del arreglo.
2. Agrega `__del__` en **Interferometro** para liberar todos los recursos pendientes cuando el objeto muera, pero implementa también un método explícito `cerrar()` que recorra las antenas y libere cada recurso; justifica por qué depender sólo de `__del__` es frágil.
3. Diseña un experimento corto que cree y destruya cientos de interferómetros con diferentes arreglos. Mide el tiempo con `time.perf_counter()` y compara el consumo si llamas o no a `cerrar()` antes de descartar la instancia.

2. Uso de `dataclass` y `field`

Contexto: describir metadatos de observaciones (experimento, frecuencia, ancho de banda, antenas activas).

1. Declara `@dataclass Observacion` con atributos inmutables (`frozen=True`) para la configuración básica.
2. Usa `field(default_factory=list)` para almacenar antenas activas y agrega métodos que verifiquen que cada antena existe en un catálogo predefinido.
3. Define un método `registrar_bandas()` que reciba una lista de bandas y valide que no haya solapamientos. Implementa pruebas unitarias rápidas con `pytest` o `unittest`.
4. Extiende la clase con `field(init=False)` para calcular un identificador hash a partir de la fecha y frecuencia central. Explica los pasos de inicialización en tu bitácora.

3. Overriding de operadores

Contexto: combinar mediciones de visibilidades complejas entre antenas.

1. Define una clase `Visibilidad` con atributos `real`, `imag` y `peso`.
2. Sobrescribe `__add__` para combinar visibilidades ponderando por `peso`. Incluye validaciones para frecuencias incompatibles.
3. Implementa `__mul__` para escalar la visibilidad por un factor real.
4. Sobrescribe `__repr__` para mostrar el valor complejo y el peso. Diseña un script que lea datos sintéticos y pruebe las operaciones, comparándolas con `numpy`.
5. Reflexiona sobre cuándo es preferible usar funciones puras en vez de sobrecargar operadores.

4. Herencia y clases abstractas (con y sin `dataclass`)

Contexto: jerarquizar procesadores de correlación para distintos telescopios manteniendo contratos bien definidos.

1. Declara `ProcesadorCorrelacion` como clase abstracta (derivada de `abc.ABC`) sin `dataclass`. Define `@abstractmethod` `configurar()` y `procesar()` y agrega un método concreto `resumen()` que reporte el tipo de arreglo.
2. Implementa subclases `ProcesadorTerrestre` y `ProcesadorEspacial` que sobrescriban ambos métodos abstractos. Cada implementación debe documentar qué recursos adicionales requiere (calibración atmosférica, compensación orbital) y reutilizar `resumen()`.
3. Construye ahora una jerarquía paralela usando `@dataclass`: `NodoCorrelacion` abstracto con campos comunes (por ejemplo, `frecuencia`, `ancho_banda`) y método abstracto `capacidad_ops()`. Deriva `NodoCPU` y `NodoGPU`, utiliza `field(init=False)` para calcular `id_nodo` y demuestra cómo llamar a `super().__post_init__()`.
4. Diseña pruebas unitarias que verifiquen que instanciar las clases base lanza `TypeError`, que las subclases respetan los contratos y que la combinación de `ProcesadorCorrelacion` con `NodoGPU` mediante herencia múltiple funciona.
5. Resume en una tabla corta las ventajas/desventajas entre usar clases abstractas “manuales” vs. `dataclass` abstractas: verbosidad, validación, extensibilidad y compatibilidad con herencia múltiple. Documenta tus conclusiones en la bitácora.

5. Getters y setters con `property`

Contexto: controlar la ganancia electrónica de cadenas receptoras.

1. Implementa `CadenaReceptora` con atributo privado `_ganancia_db`.
2. Usa `@property` y `@ganancia_db.setter` para normalizar el rango permitido (por ejemplo, de -10 a $+40$ dB) y disparar eventos de recalibración.
3. Agrega `property` calculada `ganancia_lineal` que derive el valor lineal a partir de dB.
4. Escribe pruebas donde series de ajustes fuera de rango generen `ValueError`. Explica paso a paso la propagación de errores y cómo se registran en un log.
5. Analiza cómo integrar este patrón con `dataclass` usando `init=False` para campos derivados.

6. Proyecto integrador sugerido

Desarrolla un simulador simplificado de una sesión de observación:

1. Modelo de red de antenas (sección 1) que produce visibilidades sintéticas (sección 3).
2. Catálogo de observaciones y nodos de procesamiento (sección 2 y 4).
3. Control de cadenas receptoras con propiedades (sección 5).